Network Traffic Analysis System: Project Report

Computer Networks course project · Nov 10, 2026 · @hakeem

Abstract

This project builds a software system that monitors network traffic and automatically flags bottlenecks and attacks. It reads packets from a live network interface or from saved `.pcap` / `.pcapng` files, turns each packet into a row of a table, computes traffic statistics, and runs nine rule-based detectors. Three detectors find bottlenecks (link saturation, a dominant talker, TCP retransmissions); six find attacks (port scan, SYN flood, ICMP flood, ARP spoofing, brute-force login, DNS tunnelling).

The system is written in Python with Scapy and pandas, and offers a Streamlit web dashboard and a command-line tool. On one-minute test captures with seven injected events, it detected every event with the correct hosts and raised no alerts on a clean capture of the same network. A real-time monitor re-analyzes traffic every 2 seconds and reported each attack within 2.6 seconds of it starting.

Introduction

Network administrators need to know when a link is overloaded and when someone is attacking the network, and they need to know quickly. Tools like Wireshark show every packet but leave the interpretation to a human, who cannot read thousands of packets a second. This project automates that interpretation for the most common problems on a small LAN.

Objectives

1. Capture packets live from a network interface, and read saved capture files from Wireshark or tcpdump.
2. Measure the traffic: throughput over time, protocol and service mix, top talkers and top conversations.
3. Detect bottlenecks: link saturation, hosts that hog bandwidth, and TCP retransmissions caused by packet loss.
4. Detect attacks: port scans, SYN floods, ICMP floods, ARP spoofing, brute-force logins and DNS tunnelling.
5. Present results in a dashboard and a CLI, including a real-time monitor that raises alerts as they happen.

6. Verify detection accuracy with repeatable test captures that contain known events.

Scope

The system targets IPv4/IPv6 Ethernet LANs. It inspects packet headers only (Ethernet, ARP, IP, TCP, UDP, ICMP, DNS), not encrypted payloads. Detection is rule-based, with thresholds that the user can tune.

Background

Every detector in this system relies on a specific, observable property of a network protocol. This section summarizes those properties.

Packet capture

A network card normally delivers only frames addressed to its host to the operating system. A capture library (libpcap on Linux and macOS, Npcap on Windows) puts the card into promiscuous mode and copies every frame to user space, with a timestamp. Scapy, the Python library used here, reads those frames and decodes each protocol layer. Captures can be saved in the standard pcap or pcapng formats, which Wireshark and tcpdump also use.

TCP behaviour that reveals problems

- **Three-way handshake.** A client sends SYN, the server answers SYN-ACK, and the client confirms with ACK. A closed port answers RST instead.
- **Sequence numbers.** Each data segment carries the sequence number of its first byte. If the sender gets no ACK in time, it resends the segment with the same sequence number. A repeated sequence number in the same flow is therefore a retransmission.
- **Congestion.** When a link is near capacity, router queues fill, packets are dropped and TCP retransmits. High utilization and rising retransmissions together point to a bottleneck.

ARP

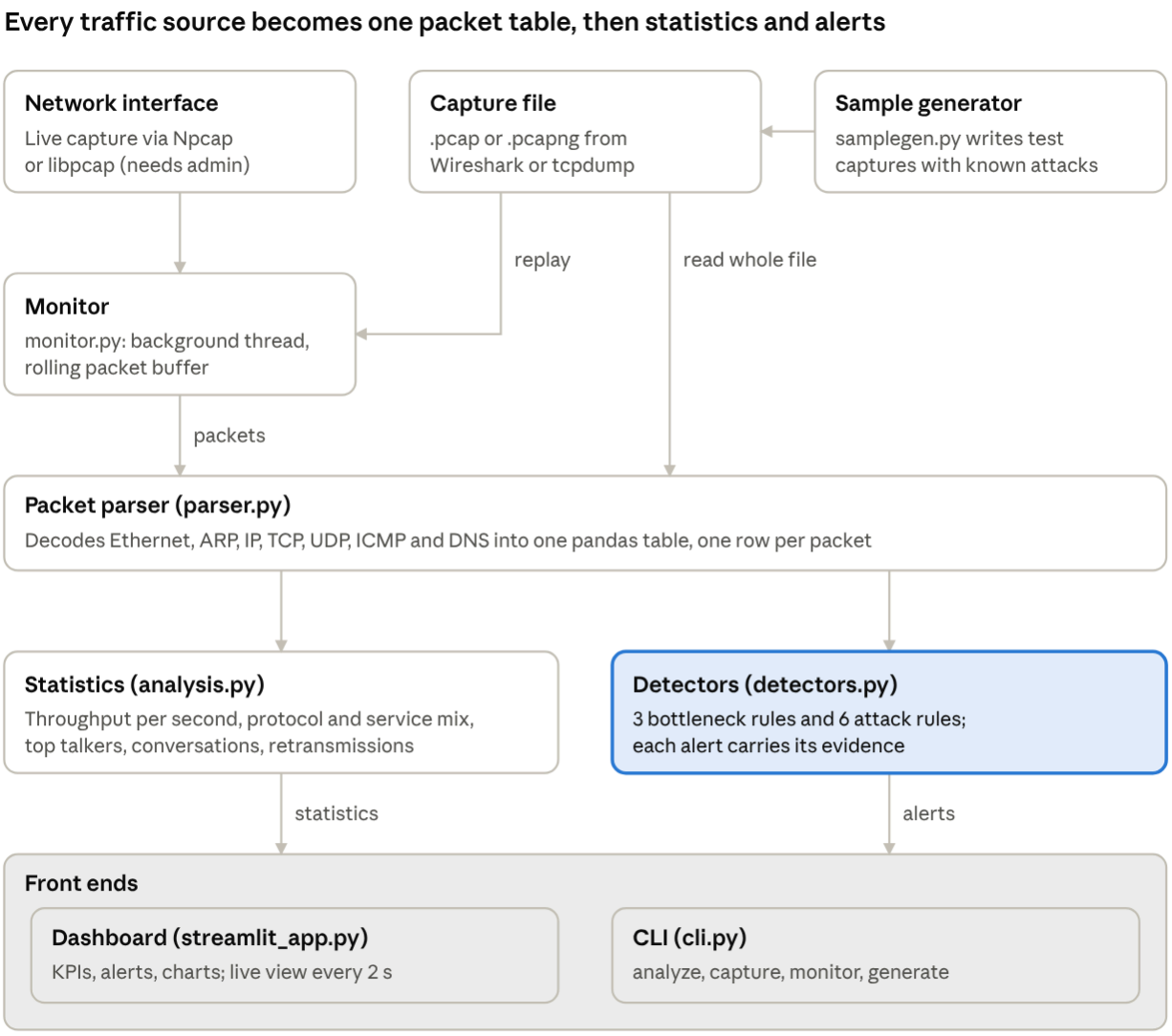
The Address Resolution Protocol maps an IP address to a MAC address on a LAN. Hosts accept ARP replies without authentication, so any host can claim to own any IP. This is the weakness that ARP spoofing exploits.

Attacks detected

Attack	Layer	How it works	What it looks like in traffic
Port scan	Transport (TCP)	Attacker sends SYNs to many ports to find open services	One source, one target, SYNs to many distinct ports; SYN-ACK from open ports, RST from closed ones
SYN flood	Transport (TCP)	Attacker sends SYNs faster than the server can handle and never completes the handshake, filling the server's backlog	Hundreds of SYNs per second to one port, often from spoofed random sources, mostly unanswered
ICMP flood	Network (ICMP)	Attacker sends echo requests at high rate to consume bandwidth and CPU	A burst of ICMP type 8 packets to one host
ARP spoofing	Data link (ARP)	Attacker tells victims that the gateway's IP is at the attacker's MAC, redirecting their traffic (man-in-the-middle)	One IP address announced by two or more MAC addresses
Brute-force login	Application (SSH, RDP, FTP and others)	Attacker tries many passwords, reconnecting every few guesses	One source opens dozens of short connections to one login port
DNS tunnelling	Application (DNS)	Malware encodes stolen data in query names under the attacker's domain; firewalls usually let DNS through	One host sends many unique, long names under one domain

System design

All three traffic sources are converted into the same packet table, so the statistics and detectors work identically on live traffic, saved files and test captures.



system architecture · 3 sources, 1 packet table, 2 analysis stages, 2 front ends

The monitor sits between a source and the parser only in real-time mode; a whole file is parsed in one pass. The detectors are the core of the system and are kept separate from the front ends, so the dashboard and the CLI always give the same results.

Implementation

The system is about 1,600 lines of Python 3.11 (plus 120 lines of tests), split into a reusable package (`nta/`) and two front ends.

Tools and technologies

Tool	Version	Used for
Python	3.11	Implementation language

Tool	Version	Used for
Scapy	2.8	Live capture, packet decoding, reading and writing pcap files, building test packets
pandas	3.1	Packet table, grouping and time-binned statistics
Streamlit	1.67	Web dashboard, including ECharts charts and auto-refreshing live view
pytest	8	Automated tests
Npcap / libpcap	n/a	Capture driver (only needed for live capture)

Packet parser (`nta/parser.py`)

Each packet becomes one record with: timestamp, frame length, source and destination MAC and IP, protocol (TCP, UDP, ICMP, ARP), ports, TCP flags, sequence number, payload length, ICMP type, ARP opcode and sender MAC, and the DNS query name. The application service (HTTP, HTTPS, DNS, SSH and 16 others) is inferred from the well-known port of either endpoint. Records are collected into a pandas DataFrame, and a relative time column `t` (seconds since the first packet) is added. Files are streamed with `PcapReader`, so pcap and pcapng are both supported without loading the whole file at once.

Statistics (`nta/analysis.py`)

- **Throughput:** packets and bytes per 1-second bin, converted to Mbit/s, with empty seconds filled with zero.
- **Breakdown:** packets, bytes and share per protocol and per service.
- **Top talkers:** bytes sent and received per host, with each host's share of all bytes.
- **Conversations:** host pairs (direction-independent) ranked by bytes, with duration and average rate.
- **Retransmissions:** TCP data segments whose (source, destination, ports, sequence number) was already seen.

Detectors (`nta/detectors.py`)

Nine functions each take the packet table and a `Thresholds` object and return a list of `Alert`s. An alert records its category, kind, severity, source, target, time window and a plain-English explanation. The detection rules are described in the next section.

Real-time monitor (`nta/monitor.py`)

A `Monitor` collects packets on a background thread into a bounded buffer (200,000 packets by default), so memory stays fixed on long runs. It has two feeds:

- **Interface:** Scapy's `AsyncSniffer` captures from a network card.
- **Replay:** a saved capture is fed in at its recorded pace (1x to 10x), which behaves exactly like live traffic and needs no capture rights. The file is decoded once up front and its rows are released on schedule, so even a 10x replay uses little CPU.

The dashboard or CLI calls `snapshot()` every 2 seconds and re-runs the full analysis on everything received so far. Only packets that arrived since the previous refresh are converted to rows, so a refresh takes under 0.4 s.

Dashboard (`streamlit_app.py`)

The dashboard has three sources (Sample, Upload, Live) and shows:

1. Eight KPI cards: packets, data, duration, hosts, average and peak throughput, attacks and bottlenecks.
2. An alert list with severity badges and explanations, downloadable as CSV.
3. A throughput timeline with the link-capacity line and alert periods shaded.
4. Traffic by service (donut chart) and top talkers (bar chart).
5. A top conversations table and a filterable packet explorer.

All thresholds can be changed in the sidebar and the results update immediately. In Live mode, the view refreshes itself every 2 seconds without reloading the page.

Command-line tool (`cli.py`)

Command	What it does
`generate`	Writes the three sample captures (normal, demo, advanced) to `samples/`
`analyze <file>`	Prints the summary, protocol breakdown, top talkers and alerts for a capture file; with --json PATH it also saves them as JSON for other tools
`interfaces`	Lists network interfaces available for capture
`capture -d 30 -o out.pcap`	Captures for a fixed time, saves the pcap, then analyzes it
`monitor [--replay file]`	Monitors continuously and prints each alert the moment it first appears

Test data generator (`nta/samplegen.py`)

Uses Scapy to build realistic packets for a small office LAN (192.168.1.0/24): DNS lookups, complete TCP sessions with handshake, data, ACKs and FIN, and routine ARP. It can inject each of the six attacks and a congested download. A fixed random seed makes every generated file identical, so test results are repeatable.

Detection methodology

Each detector applies one rule to the packet table; an alert fires when the rule's threshold is crossed. The default thresholds below suit a small LAN on a 10 Mbit/s uplink, and every one can be changed in the dashboard.

Bottleneck rules

Alert	Severity	Rule (default)	Rationale
Link saturation	High if peak reaches capacity, else Medium	Any 1-second bin above 80% of link capacity (10 Mbit/s); consecutive seconds merge into one episode	Queues build and latency rises before a link is fully used. The alert names the flow that carried the most bytes during the episode, which is the likely cause.

Alert	Severity	Rule (default)	Rationale
Dominant talker	Medium	One host sends 50% or more of all bytes, once at least 1 MB has been captured	One host can starve the others on a shared link. The 1 MB minimum avoids false alarms in the first seconds of a live capture.
High retransmission rate	Medium	3% or more of all TCP data segments are retransmissions (at least 100 segments)	TCP retransmits after loss, and loss on a busy path means congestion. Healthy networks usually stay well below 1%.
Lossy TCP flow	Low	One flow of 100 or more segments has 3% or more retransmitted	Points to the specific connection that suffers.

Attack rules

Alert	Severity	Rule (default)	Rationale
Port scan	High	One source sends SYN-only packets to 20 or more distinct ports of one target	A normal client contacts one or two service ports per server. The alert also lists open ports, found from the target's SYN-ACK replies.
SYN flood	High	100 or more SYNs per second to one host and port	Only the burst seconds are reported, so ordinary connections to the same port don't stretch the alert. It reports the share of SYNs answered with SYN-ACK, and flags spoofing when there are more than 50 distinct sources.
ICMP flood	Medium	50 or more ICMP echo requests per second to one host	Normal ping sends one request per second.

Alert	Severity	Rule (default)	Rationale
ARP spoofing	High	One IP address appears in ARP replies with two or more different sender MACs	A legitimate IP has one MAC. Two MACs for the gateway's IP is the signature of a man-in-the-middle attack.
Brute-force login	High	One source opens 20 or more connections to one login service (SSH, RDP, FTP, Telnet, POP3, IMAP, MySQL, VNC)	A person logs in once and keeps the session; a guessing tool reconnects every few passwords. The alert reports attempts per minute. A port scan touches each port once, so it does not trigger this rule.
DNS tunnelling	High	One host queries 30 or more unique names under one domain, averaging 40 or more characters	Normal lookups reuse a few short names that are then cached. Tunnels encode data in long one-off names; the alert estimates how much data left the network.

Why rule-based?

Rules are transparent: every alert states the exact numbers that triggered it, such as "SYN probes to 500 distinct ports in 2.5s". That makes results easy to verify and explain, and thresholds easy to adapt to a different network. Machine-learning detectors can catch subtler anomalies, but they need labelled training data and their decisions are harder to justify. This is noted as future work.

Testing and results

The system detected all 7 injected events across two attack captures (10 alerts in total) with the correct hosts, raised zero alerts on clean traffic, and in real-time mode reported every attack within 2.6 seconds of its start.

Test setup

Three one-minute captures of a simulated office LAN were generated with a fixed seed: gateway 192.168.1.1, web server 192.168.1.20, five clients 192.168.1.10 to .14, browsing four internet servers.

- **normal.pcap** (4,898 packets, 4.3 MB): DNS lookups, HTTPS sessions and routine ARP only. Peak throughput 2.2 Mbit/s.

- **demo.pcap** (18,070 packets, 14.2 MB): the same baseline plus five injected events.
- **advanced.pcap** (6,138 packets): the same baseline plus SSH password guessing and DNS tunnelling.

Detection results

Injected event (start)	Alert raised	Evidence reported by the system
TCP SYN scan of ports 1 to 500 on .20 by .66 (t = 9.8 s)	Port scan, High	"SYN probes to 500 distinct ports in 2.5s. Open ports revealed: 22, 80, 443."
.66 claims the gateway's IP in ARP replies every 2 s (t = 18 s)	ARP spoofing, High	"192.168.1.1 claimed by 2 MAC addresses: de:ad:be:ef:00:66 (105 replies), 00:11:22:33:44:01 (6 replies)"
11 Mbit/s download to .12 over a 10 Mbit/s link with 6% loss (t = 26 s)	Link saturation, High; Dominant talker, Medium; High retransmission rate, Medium; Lossy TCP flow, Low	"6s above 80% of the 10 Mbit/s link (peak 14.4 Mbit/s). 151.101.1.69 → 192.168.1.12 caused 92% of it." Overall retransmissions 3.5%; that flow 5.1%
2,500 spoofed-source SYNs to .20:80 in 4 s (t = 39.8 s)	SYN flood, High	"2,500 SYNs from 2,500 sources, peak 625/s; only 10% answered with SYN-ACK."
400 large pings from .77 to .20 in 2 s (t = 49.8 s)	ICMP flood, Medium	"400 echo requests (0.42 MB), peak 200/s."
120 SSH login attempts on .20:22 from 203.0.113.50 in 30 s (advanced.pcap, t = 7.8 s)	Brute-force login, High	"120 SSH connection attempts in 30s (242 per minute), typical of password guessing."
200 base32-encoded queries under tunnel.example from .77 in 40 s (advanced.pcap, t = 14.8 s)	DNS tunnelling, High	"200 unique names under tunnel.example, average 72 characters (about 11.4 KB of encoded data)"
None (normal.pcap)	No alerts	All rates below thresholds; retransmissions 0.3%

No false positives were raised in any capture: every alert in demo.pcap and advanced.pcap maps to an injected event, and the demo attacks did not trigger the two newer rules. The automated test suite (10 tests, `python -m pytest`) checks these results on every run.

Real-time monitor

demo.pcap was replayed through the monitor at its real pace (1x), with the analysis refreshing every 2 s.

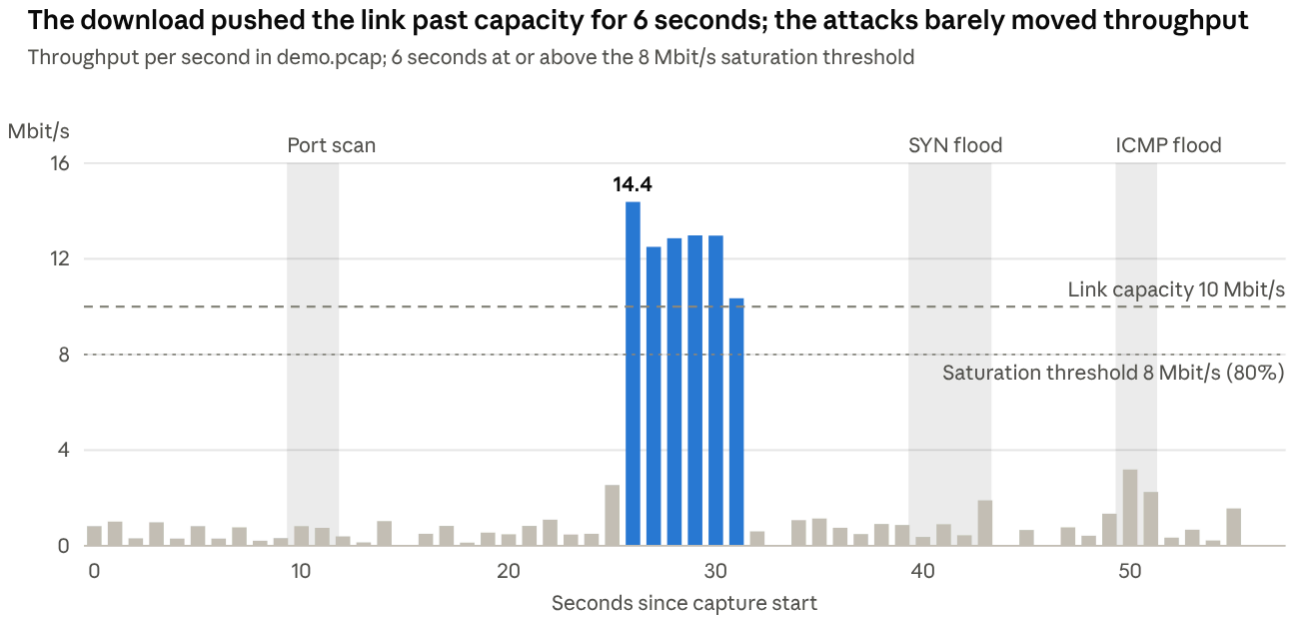
Event	Started (s)	Alert shown (s)	Delay (s)
Port scan	9.8	10.4	0.6
ARP spoofing	17.8	18.8	1.0
Link saturation	26.0	27.4	1.4
SYN flood	39.8	40.8	1.0
ICMP flood	49.8	52.4	2.6

The delay is the time an attack needs to cross its threshold plus at most one 2-second refresh. The ICMP flood took longest: its first second held only 40 pings, below the 50-per-second threshold, so it crossed the threshold at about 50.3 s and appeared at the next refresh.

Performance

On the test laptop, parsing ran at about 3,900 packets per second, and running all nine detectors on the 18,070-packet capture took 125 ms. An early version took 781 ms because the port scan and SYN flood detectors looped in Python over every source; filtering with vectorized pandas counts first made it 7 times faster (116 ms with the original seven detectors).

Throughput during the demo capture



measured from samples/demo.pcap · 58 one-second bins · link capacity 10 Mbit/s

Only the congested download shows up in throughput. The scan, the SYN flood and the ICMP flood each stay under 3.2 Mbit/s, which is why they need protocol-level detectors rather than a bandwidth alarm.

Output screenshots

The screenshots below were taken from the running dashboard on 10 November 2026. Full-resolution copies are in the project's `docs/screenshots/` folder.

Figure 1: Dashboard on the demo capture

The eight KPI cards sit above the alert list; each alert names the hosts, the time window and the evidence that triggered it.

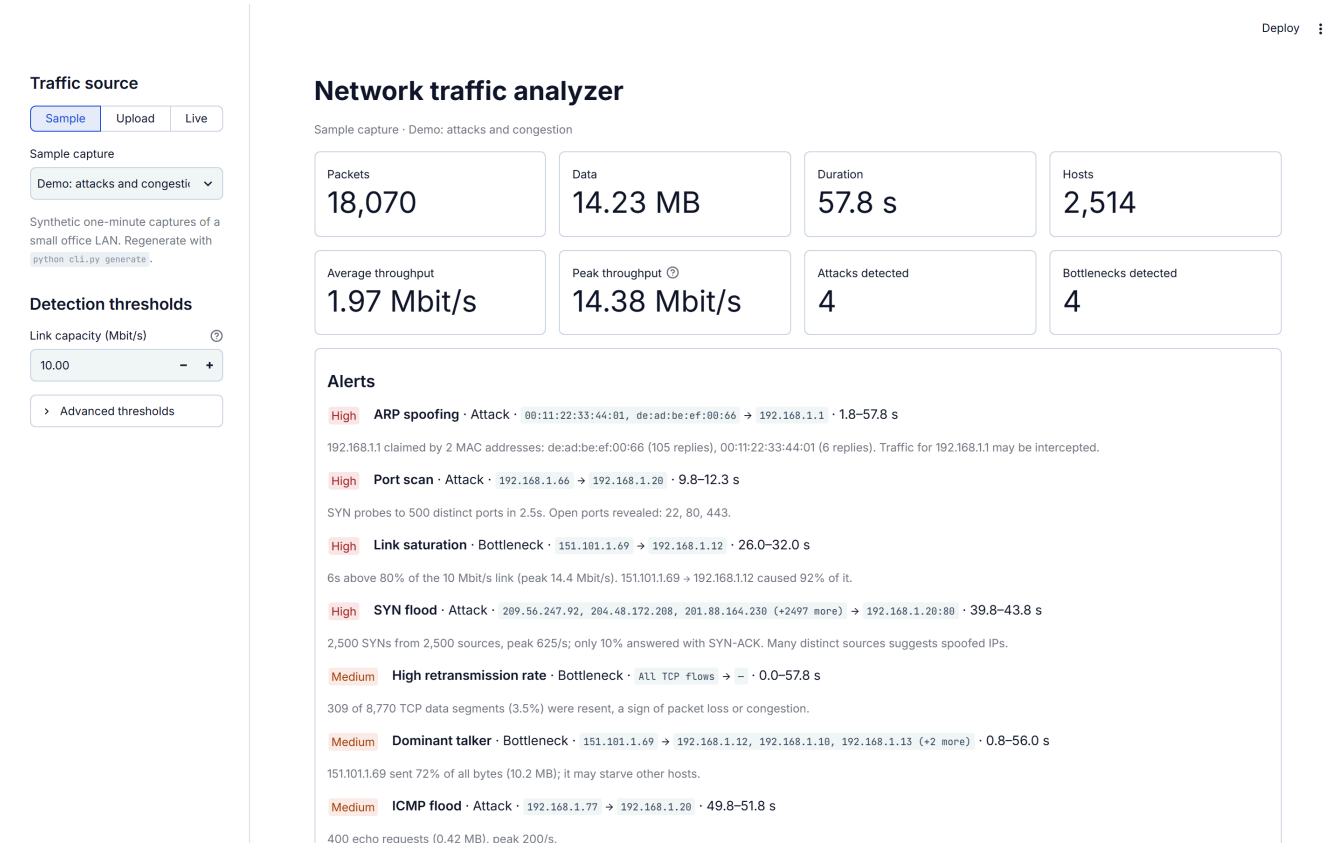


Figure 2: Throughput timeline

Throughput in Mbit/s and packets per second, with the 10 Mbit/s link capacity dashed and short-lived alerts shaded. The SYN flood shows as a spike in packets per second but not in Mbit/s.



Figure 3: Alerts on the advanced capture

The two newer rules catch SSH password guessing and DNS tunnelling, and nothing else fires on this capture.



Figure 4: Live monitor during a 5x replay

Taken about 8 seconds after pressing Start, with 40 seconds of the demo capture played. The port scan, ARP spoofing, link saturation and the start of the SYN flood are already reported; the view refreshes every 2 seconds.

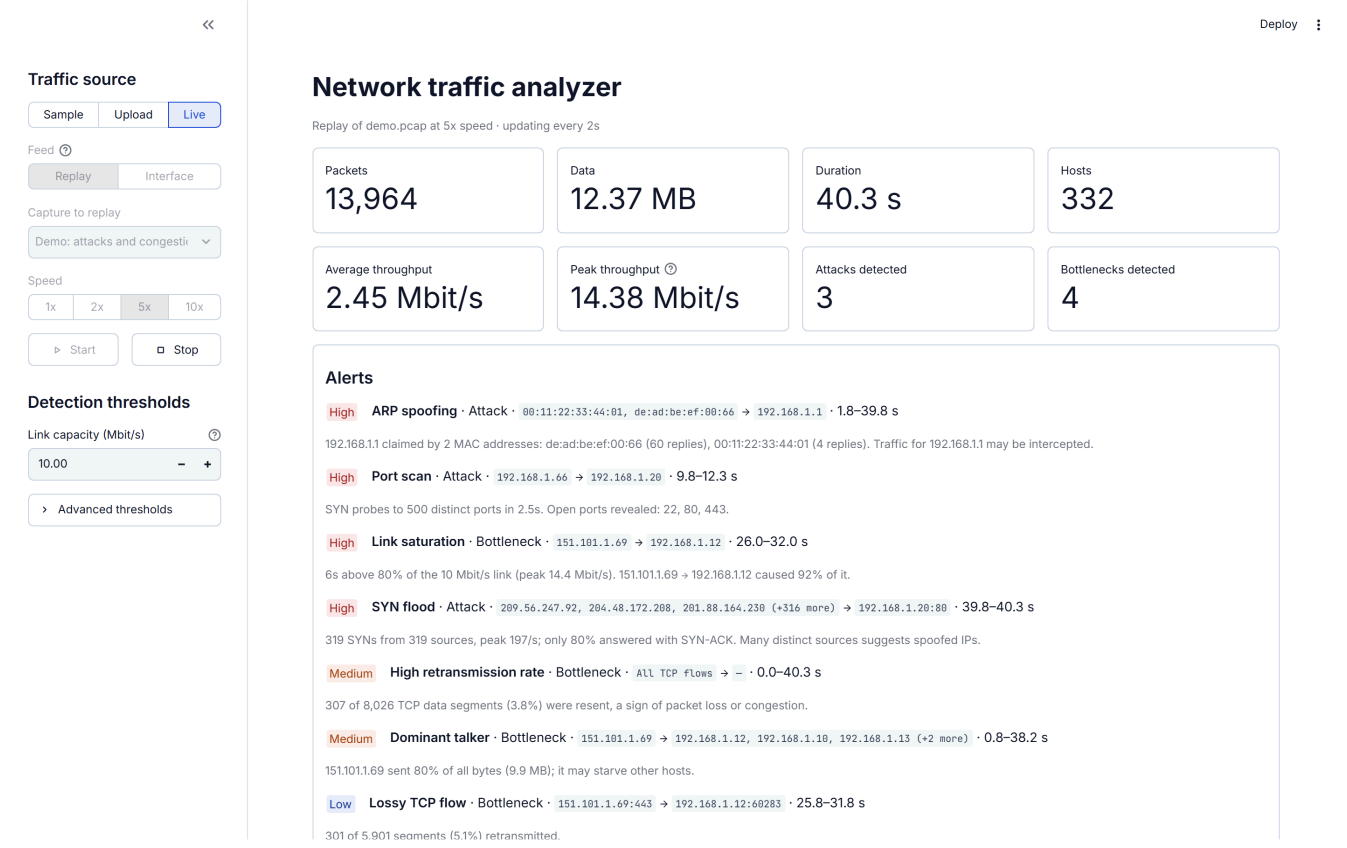
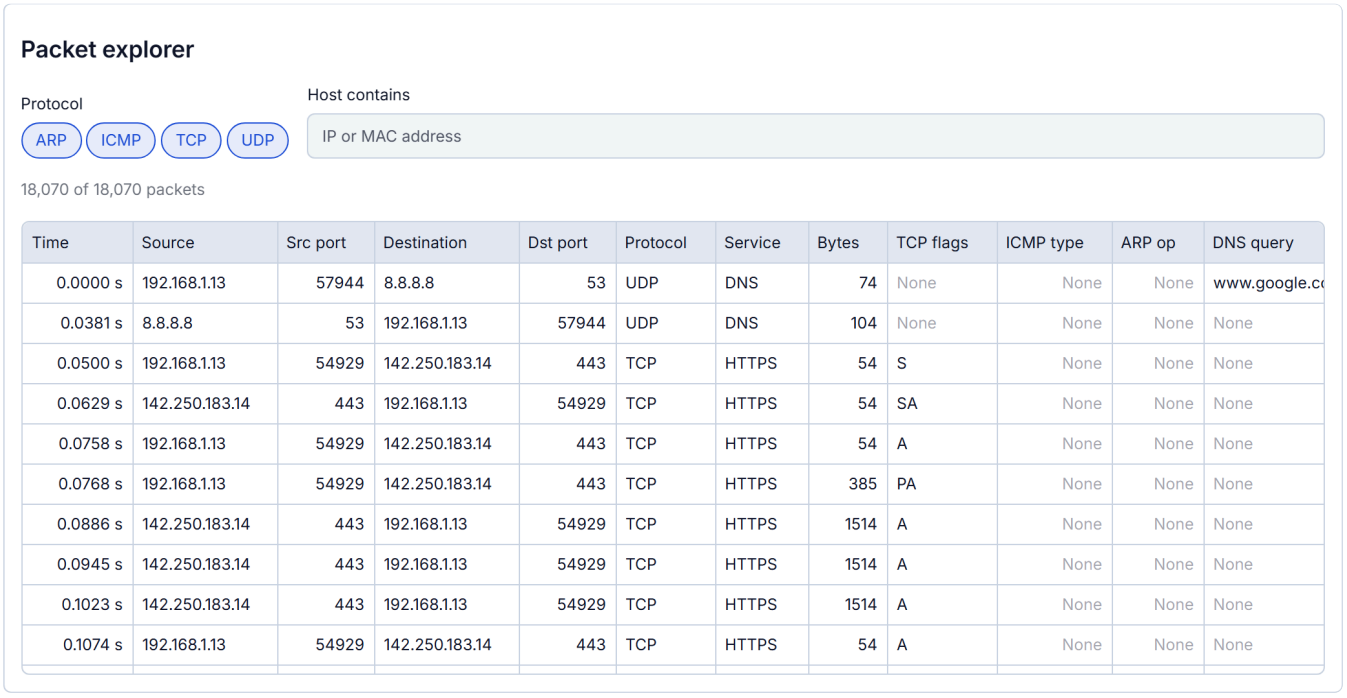


Figure 5: Packet explorer

Individual packets with protocol and host filters, showing ports, TCP flags, ICMP types, ARP operations and DNS query names.



Packet explorer

Protocol

ARP ICMP TCP UDP

Host contains

IP or MAC address

18,070 of 18,070 packets

Time	Source	Src port	Destination	Dst port	Protocol	Service	Bytes	TCP flags	ICMP type	ARP op	DNS query
0.0000 s	192.168.1.13	57944	8.8.8.8	53	UDP	DNS	74	None	None	None	www.google.c
0.0381 s	8.8.8.8	53	192.168.1.13	57944	UDP	DNS	104	None	None	None	None
0.0500 s	192.168.1.13	54929	142.250.183.14	443	TCP	HTTPS	54	S	None	None	None
0.0629 s	142.250.183.14	443	192.168.1.13	54929	TCP	HTTPS	54	SA	None	None	None
0.0758 s	192.168.1.13	54929	142.250.183.14	443	TCP	HTTPS	54	A	None	None	None
0.0768 s	192.168.1.13	54929	142.250.183.14	443	TCP	HTTPS	385	PA	None	None	None
0.0886 s	142.250.183.14	443	192.168.1.13	54929	TCP	HTTPS	1514	A	None	None	None
0.0945 s	142.250.183.14	443	192.168.1.13	54929	TCP	HTTPS	1514	A	None	None	None
0.1023 s	142.250.183.14	443	192.168.1.13	54929	TCP	HTTPS	1514	A	None	None	None
0.1074 s	192.168.1.13	54929	142.250.183.14	443	TCP	HTTPS	54	A	None	None	None

User guide

Installation

1. Install Python 3.10 or newer.
2. In the project folder, create an environment and install the dependencies:

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
```

3. For live capture only: on Windows, install Npcap with "WinPcap API-compatible mode" ticked and run the terminal as Administrator. On Linux or macOS, run with `sudo`. File analysis and replay need neither.

Running the dashboard

```
streamlit run streamlit_app.py
```

On Windows you can instead double-click `run_dashboard.bat`; on first run it creates the environment and installs the packages.

The dashboard opens at http://localhost:8501. In the sidebar, choose a source:

- **Sample:** the built-in normal, demo or advanced capture.
- **Upload:** any `.pcap`, `.pcapng` or `.cap` file saved from Wireshark or tcpdump.
- **Live:** the real-time monitor. Choose **Replay** to stream a sample capture at 1x to 10x speed, or **Interface** to watch a network card; then press **Start**. When stopped, the collected packets can be downloaded as a pcap.

Set **Link capacity** to the bandwidth of the monitored link; the other thresholds are under **Advanced thresholds**.

Using the CLI

```
python cli.py generate
python cli.py analyze samples/demo.pcap --capacity 10
python cli.py analyze samples/advanced.pcap --json report.json
python cli.py interfaces
python cli.py capture -d 30 -f "tcp or udp" -o mine.pcap
python cli.py monitor --replay samples/demo.pcap --speed 5
python cli.py monitor -i "Wi-Fi" -o session.pcap
python -m pytest
```

Limitations and future work

Limitations

- **Fixed thresholds.** A flood just below the per-second thresholds is not flagged, and a busy network may need higher thresholds to avoid false alarms. Port scans are counted over the whole capture, so slow scans are caught in files; the live monitor only sees scans within its rolling buffer of 200,000 packets.
- **Header-only inspection.** Payloads, which are mostly encrypted (HTTPS), are not inspected, so application-layer attacks are out of scope. Services are inferred from port numbers, which can be wrong for non-standard ports.
- **Throughput of the parser.** Python decoding handles about 3,900 packets per second, enough for a small LAN but not for a busy gigabit link. The dashboard analyzes at most 300,000 packets per uploaded file.
- **Synthetic evaluation.** Accuracy was measured on generated traffic, where the ground truth is known. Results on real networks will depend on their traffic mix.
- **Single vantage point.** The monitor sees only traffic that reaches its interface; on a switched network that needs a mirror (SPAN) port.

Future work

1. Learn each network's normal baseline and alert on deviations, instead of fixed thresholds.
2. Keep long-term per-host counters so the live monitor also catches slow scans and slow password guessing beyond its rolling buffer.
3. Decode with a faster engine (tshark or a compiled parser) for high-speed links.
4. Send alerts by email or chat, and keep an alert history in a database.
5. Evaluate on public labelled datasets such as CIC-IDS2017.

Conclusion

The project met all six objectives. The system captures or loads traffic, measures it, and explains bottlenecks and attacks in plain language, in both a dashboard and a CLI. On test captures it detected every injected event with the correct hosts and no false positives, and in real-time mode it reported each attack within 2.6 seconds. Because every alert rests on an observable protocol behaviour (an unanswered SYN, a repeated sequence number, a second MAC for one IP), the results are easy to verify and to explain.

References

1. J. Postel, "Transmission Control Protocol", RFC 793, 1981 (updated by RFC 9293, 2022). Three-way handshake, sequence numbers and retransmission.
2. D. Plummer, "An Ethernet Address Resolution Protocol", RFC 826, 1982.
3. J. Postel, "Internet Control Message Protocol", RFC 792, 1981.
4. W. Eddy, "TCP SYN Flooding Attacks and Common Mitigations", RFC 4987, 2007.
5. J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, Pearson.
6. Scapy documentation, https://scapy.readthedocs.io
7. Npcap, packet capture library for Windows, https://npcap.com
8. pandas documentation, https://pandas.pydata.org/docs
9. Streamlit documentation, https://docs.streamlit.io

References are cited from memory and were not checked online.

Appendix A: Sample CLI output

Output of `python cli.py analyze samples/advanced.pcap`, unedited:

```
=== Summary ===
Packets: 6,138   Data: 4.44 MB   Duration: 56.0s   Hosts: 14
Throughput: avg 0.63 Mbit/s, peak 2.23 Mbit/s (link capacity 10 Mbit/s)

=== Protocols ===
proto  packets   bytes share
  TCP     5564 4.36 MB 98.1%
  UDP      562 0.09 MB  1.9%
  ARP       12 0.00 MB  0.0%

=== Top talkers (by bytes sent) ===
          host  packets_sent bytes_sent bytes_received share_sent
  151.101.1.69           878    1.26 MB        0.03 MB      28.3%
  13.107.42.14           823    1.16 MB        0.04 MB      26.1%
142.250.183.14           712    1.00 MB        0.03 MB      22.5%
104.16.132.229           536    0.75 MB        0.02 MB      16.9%
       8.8.8.8           281    0.05 MB        0.03 MB       1.2%

=== Alerts (2) ===
[High  ] Attack / Brute-force login  t=7.8-37.6s
          203.0.113.50 -> 192.168.1.20:22
          120 SSH connection attempts in 30s (242 per minute), typical of
password guessing.
[High  ] Attack / DNS tunnelling  t=14.8-54.6s
          192.168.1.77 -> tunnel.example
          200 unique names under tunnel.example, average 72 characters (about
11.4 KB of encoded data), typical of data exfiltration or a covert channel
over DNS.
```

Appendix B: Project files

Path	Contents
`nta/parser.py`	Packet decoding into a pandas table
`nta/analysis.py`	Traffic statistics
`nta/detectors.py`	The nine detection rules and their thresholds
`nta/monitor.py`	Real-time monitor: live capture or paced replay
`nta/capture.py`	Interface listing and fixed-length capture
`nta/samplegen.py`	Generator for the three test captures
`streamlit_app.py`	Web dashboard

Path	Contents
`cli.py`	Command-line tool
`tests/`	10 automated tests (pytest)
`samples/`	normal.pcap, demo.pcap, advanced.pcap
`docs/screenshots/`	Dashboard screenshots used in this report
`run_dashboard.bat`	One-click launcher for Windows
`requirements.txt`, `README.md`	Dependencies and quick-start guide